

mycv

Mathematical Foundations

A Pure-NumPy Computer Vision Library

Abodunrin Charles Toluwanimi

Department of Electrical and Electronic Engineering
Obafemi Awolowo University, Ile-Ife

An updated mathematical and implementation reference for mycv v4.2.1. This edition extends the earlier v2.0.0 and v3.0.0 notes with tracking, calibration, region geometry, neural networks, and the bundled shape classifier.

Version 4.2.1 | September 2026

Contents

Chapter
Conventions and module map
1. Point operations and colour
2. Filters and morphology
3. Geometry and calibration
4. Features and measurements
5. Detection and suppression
6. Segmentation and tracking
7. Neural networks and shape CNN
8. Numerical practice
9. API map and references
10. Normalized DLT derivation
11. RANSAC consensus
12. Harris and Hough derivations
13. Region moments
14. NCC invariance and implementation
15. Kalman filter
16. CNN layers and gradients
17. Shape CNN and data scope
18. Worked pipeline and parameters
19. Formula index
20. Function-by-function API guide

Contents and conventions

This reference follows the source tree and public API of mycv v4.2.1. The core image algorithms use NumPy and do not require OpenCV, SciPy, or scikit-image. Optional display, image I/O, and streaming features use separate packages.

Image domain: $\Omega = \{0, \dots, H-1\} \times \{0, \dots, W-1\}$
 Coordinates: $(x, y) = (\text{column}, \text{row})$; array indexing is `image[y, x]`.
 RGB shape: $(H, W, 3)$ Grayscale shape: (H, W)
 Mask foreground: $M[y, x] > 0$
 Box: $(y1, x1, y2, x2)$, exclusive bottom-right; `crop = image[y1:y2, x1:x2]`

Module map

core/color: pointwise maps, luminance, histogram and HSV. filters/morphology: discrete operators, set morphology and connected components. geometry/calibration: interpolation, projective maps and homography estimation. features/detection: local moments, shape descriptors, template matching and overlap. tracking: segmentation, state estimation and data association. nn/shape_cnn: reverse-mode differentiation and learned shape classification.

Coordinate order and endpoint conventions matter when data moves between modules. In particular, boxes use row-first order and exclusive end coordinates.

1. Point operations and colour

RGB to grayscale

For RGB components in $[0, 255]$, the library applies the BT.601-style luminance map. It computes in floating point, clips, rounds, and returns uint8. This is a luma-like grayscale conversion, not a camera-profile-aware colourimetric transform.

$$Y = 0.299 R + 0.587 G + 0.114 B$$

Threshold

A global inclusive threshold partitions grayscale values into foreground and background. A single threshold may be inadequate under changing illumination.

$$T_{\text{tau}}(I)[y, x] = 255 \text{ if } I[y, x] \geq \text{tau, otherwise } 0$$

Histogram equalization

Let $h(k)$ count pixels at intensity k and $N=H W$. The empirical probability mass and cumulative distribution define an intensity lookup table.

$$\begin{aligned} p(k) &= h(k)/N & F(k) &= \sum_{j=0..k} p(j) \\ L(k) &= \text{round}(255 F(k)) & I_{\text{eq}}[y, x] &= L(I[y, x]) \end{aligned}$$

RGB to HSV and masks

Normalize RGB to $[0, 1]$. Let $V=\max(R, G, B)$, $m=\min(R, G, B)$, and $\Delta=V-m$. Saturation is Δ/V for $V>0$ and zero otherwise. Hue is selected piecewise from the maximum channel and lies in $[0, 360)$; saturation and value lie in $[0, 1]$. Achromatic pixels use hue zero by convention.

$$\begin{aligned} S &= \Delta/V \text{ when } V>0, \text{ else } 0 \\ H &= 60 * ((G-B)/\Delta \bmod 6) \text{ if } V=R \end{aligned}$$

```
H = 60*((B-R)/Delta + 2)    if V=G
H = 60*((R-G)/Delta + 4)    if V=B
```

Colour masks use inclusive HSV range checks. The hue-wrap helper handles intervals crossing 0/360. Adaptive masking uses saturation and value thresholds for near-achromatic samples, where hue is numerically unstable.

2. Filters and morphology

Discrete convolution

The implementation performs true convolution by reflecting the kernel along both axes. Same mode uses zero padding and preserves image shape; valid mode uses no padding. A strided patch view and einsum contract each patch with the kernel.

$$(I*K)[y,x] = \sum(i,j) I[y-i,x-j] K[i,j]$$

Sobel gradients

Sobel combines a central finite difference in one direction with smoothing perpendicular to it. The API returns raw gradient maps and direction, plus magnitude normalized to the current image maximum.

```
Kx = [-1 0 1; -2 0 2; -1 0 1]    Ky = transpose(Kx)
Gx = I*Kx    Gy = I*Ky
Magnitude = sqrt(Gx^2 + Gy^2)    angle = atan2(Gy,Gx)
```

Binary and grayscale morphology

Represent the nonzero mask pixels by a set F and the structuring element by B . Dilation is an existential neighborhood reduction; erosion is a universal reduction.

```
F dilated by B = {z: exists b in B with z-b in F}
F eroded by B  = {z: for all b in B, z+b is in F}
Opening = erosion then dilation
Closing = dilation then erosion
```

Constant padding treats pixels outside the image as background for dilation and as foreground for erosion. Grayscale dilation is a sliding maximum padded by negative infinity.

Connected components

A raster scan assigns provisional labels using 4- or 8-neighbor adjacency. Union-find resolves label equivalences; a second pass maps roots to consecutive component IDs. This component algorithm uses sequential loops. Component properties include area, exclusive box, centroid, and optional mean colour.

3. Geometry and calibration

Bilinear interpolation

At a fractional source coordinate, use the four neighboring pixels and separable linear weights. The weights are nonnegative and sum to one; coordinates outside the image clamp to its nearest edge.

$$I(x_s, y_s) = (1-a)(1-b) I[y_0, x_0] + a(1-b) I[y_0, x_1] \\ + (1-a)b I[y_1, x_0] + ab I[y_1, x_1]$$

Rotation and perspective mapping

Rotation about the image center is an affine homogeneous transform. To avoid holes, each destination pixel is mapped backward through the inverse transform and sampled. A homography maps points up to projective scale; inverse mapping followed by division by the homogeneous coordinate creates perspective effects.

```
M = T(center) R(theta) T(-center)
R(theta) = [cos(theta), -sin(theta); sin(theta), cos(theta)]
(u,v,w) = inverse(H) * (x_destination,y_destination,1)
(x_source,y_source) = (u/w,v/w)
```

Normalized DLT

Each point correspondence contributes two homogeneous equations in the nine entries of H . Hartley normalization centers each point set and scales its mean radial distance to $\sqrt{2}$. SVD obtains the least-squares null-space vector, and the estimate is denormalized.

```
A h = 0; h is the right singular vector for the smallest singular value
H = inverse(T_destination) * H_normalized * T_source
```

RANSAC and reprojection error

RANSAC samples four correspondences per candidate, measures Euclidean reprojection errors, keeps the largest inlier set, and refits with all selected inliers. It is robust to outliers when a usable consensus exists, but does not automatically fix degenerate or poorly distributed points.

```
e_i = norm( project(H * [x_i,y_i,1]) - [xprime_i,yprime_i] )
inlier_i = (e_i < threshold)
```

4. Features and object measurements

Harris corners

Local weighted products of image gradients form a 2 by 2 structure tensor. Its eigenvalues distinguish corner-like regions, which vary in two directions, from edge-like regions.

```
M = [G_sigma * Ix^2, G_sigma * IxIy;
      G_sigma * IxIy, G_sigma * Iy^2]
R = det(M) - k * trace(M)^2
```

Hough lines

Each edge pixel votes for sampled lines in polar normal form. Accumulator peaks are line hypotheses; nearby peaks are clustered to estimate a count. Resolution and peak thresholds affect the result.

```
rho = x cos(theta) + y sin(theta)
```

Moments and shape descriptors

Foreground coordinates define area, centroid, central moments, orientation, and eccentricity. The exclusive bounding box has width $x_2 - x_1$ and height $y_2 - y_1$. The implementation reports true foreground pixel area separately from rectangle area.

```
A = count(foreground pixels)
centroid = (sum(x)/A, sum(y)/A)
phi = 0.5 atan2(2 mu11, mu20-mu02)
eccentricity = sqrt(1 - lambda2/lambda1)
```

```

extent = A/(box_width*box_height)
circularity = min(1, 4 pi A / perimeter^2)
solidity = A / convex_hull_area

```

The heuristic calls a region a circle only when circularity ≥ 0.85 and extent ≤ 0.88 ; otherwise aspect ratio separates square-like from rectangular shapes. Colour is selected as the nearest of six fixed RGB reference colours. These discrete measurements depend on mask quality and object scale.

5. Detection and non-maximum suppression

Normalized cross-correlation

For each valid image patch, NCC is Pearson correlation between the zero-mean patch and zero-mean template. It lies in $[-1,1]$ and is invariant to additive offsets and positive contrast scaling in the non-degenerate case. Flat patches or templates score zero.

$$\text{NCC}(x,y) = \frac{\text{mean}((P - \text{mean}(P)) * (T - \text{mean}(T)))}{(\text{std}(P) * \text{std}(T))}$$

The response map gives a score at each top-left patch location. A memory guard limits patch-statistic temporaries. Multi-scale matching searches a Gaussian pyramid and maps candidate locations back to the original scale.

Gaussian pyramid

```

G = outer([1,4,6,4,1],[1,4,6,4,1]) / 256
I[level+1] = (I[level] * G)[::2,::2]

```

IoU and NMS

Exclusive-coordinate boxes make width and height direct coordinate differences. Greedy non-maximum suppression keeps the highest-score box and discards remaining boxes whose overlap exceeds the chosen IoU threshold.

```

IoU(A,B) = area(A intersect B) / area(A union B)
area(A intersect B) = max(0, y2I-y1I) * max(0, x2I-x1I)

```

6. Segmentation and tracking

Motion and centroid

Frame differencing widens unsigned inputs before subtraction to prevent uint8 wraparound. The motion mask marks pixels whose absolute difference meets the threshold. A binary mask centroid is the arithmetic mean of foreground coordinates; an empty mask returns None.

```

D_t[y,x] = abs(I_t[y,x] - I_(t-1)[y,x])
M_t[y,x] = 255 if D_t[y,x] >= tau, otherwise 0
centroid = (sum(x)/A, sum(y)/A)

```

Kalman centroid tracking

The filter state contains image position and velocity. A constant-velocity transition predicts the next state; position measurements update that prediction. Covariance correction uses the Joseph form and linear solves for numerical stability.

```

state = [x, y, vx, vy]^T

```

```

F(dt) = [1 0 dt 0; 0 1 0 dt; 0 0 1 0; 0 0 0 1]
state_pred = F state      P_pred = F P F^T + Q
K = P_pred H^T (H P_pred H^T + R)^-1
state = state_pred + K (measurement - H state_pred)

```

Optional Mahalanobis gating rejects implausible measurements, and confidence can adjust measurement noise. Multi-object tracking assigns detections using spatial distance and manages persistent IDs, missed frames, and track creation/deletion; it is a lightweight association method, not a global optimal solver.

HSV colour segmentation

Colour tracking thresholds hue, saturation, and value. Hue-wrap segmentation handles reds that span the angular boundary. Adaptive segmentation switches to saturation/value thresholds for near-neutral samples.

7. Neural networks and the shape CNN

Reverse-mode differentiation

The NumPy neural-network module includes convolution, pooling, dense, and ReLU layers. For a dense affine layer, reverse-mode derivatives are:

```

Z = X W + b; let delta = dL/dZ
dL/dW = X^T delta
dL/db = sum(delta over batch)
dL/dX = delta W^T
ReLU(a) = max(0,a); derivative = 1[a>0]

```

Softmax cross-entropy and SGD

```

softmax(z)_c = exp(z_c) / sum_j exp(z_j)
L = -sum_c y_c log(softmax(z)_c)
dL/dz = softmax(z) - y
parameter <- parameter - learning_rate * gradient

```

Bundled shape CNN

The trained classifier accepts a non-empty cropped binary mask and predicts Circle, Square, or Rectangle. The result includes a label, confidence, and per-class probabilities. Blank masks raise ValueError. It augments the geometric heuristic; it is not a general object detector.

Version 4.2.1 retrained with the full square rotation range and applied degradation augmentation before resizing. Bundled metadata records training configuration and a measured synthetic stress-test result; that score is not a guarantee for arbitrary real camera data.

8. Numerical practice and limitations

- Cast unsigned image arrays before signed differences or products; uint8 arithmetic can wrap.
- Use floating-point intermediates for correlations, moments, projective transforms, and covariance updates.
- Guard divisions at achromatic HSV pixels, flat NCC patches, empty masks, and projective points near infinity.

- Stride-based views avoid copying patches, but reductions and elementwise products can allocate large temporary arrays.
- Connected components, contour tracing, RANSAC, and greedy suppression include sequential steps; not all operations are loop-free.
- Results depend on encoding, thresholds, scale, mask quality, noise, and camera geometry. The package exposes algorithmic building blocks rather than automatic parameter selection.

The neural-network backward passes are covered by finite-difference gradient checks in the project tests. Other tests exercise image, tracking, detection, and calibration conventions. Python docstrings remain authoritative for exact parameter and return-value details.

9. API map and references

Main modules

core: `rgb_to_grayscale`, `threshold`. color: `histogram_equalize`, `rgb_to_hsv`. filters: `convolve2d`, `Sobel`. morphology: binary and grayscale operations, `components`. geometry: `interpolation`, `rotation`, `perspective`. warp. features: `Harris`, `Hough`, region metrics and heuristic shape classification. detection: `NCC`, `pyramids` and `NMS`. tracking: `masks` and `Kalman` trackers. calibration: `normalized DLT` and `RANSAC`. nn/shape_cnn: NumPy layers and bundled classifier. streaming: optional PyAV `StreamReader`.

References

- [1] ITU-R Recommendation BT.601-7, Studio Encoding Parameters of Digital Television, 2011. [2] C. Harris and M. Stephens, A combined corner and edge detector, *Alvey Vision Conference*, 1988. [3] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2004.
- [4] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018. [5] J. P. Lewis, Fast normalized cross-correlation, *Vision Interface*, 1995. [6] C. E. Shannon, Communication in the presence of noise, *Proceedings of the IRE*, 1949. [7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.

Source, tests, bundled model metadata, and release history: <https://github.com/charlie1/mycv2>

10. Derivation: normalized DLT in detail

A planar homography has eight independent degrees of freedom because multiplying its 3 by 3 matrix by any nonzero scalar gives the same projective map. Four point correspondences provide the minimum eight independent scalar constraints, although noisy data should use more points.

$$\begin{aligned} p &= [x, y, 1]^T; \quad p_{\text{prime}} = [x_p, y_p, 1]^T; \quad p_{\text{prime}} \text{ cross } (H p) = 0 \\ [-x, -y, -1, \quad 0, 0, 0, \quad x_p x, x_p y, x_p] \quad h &= 0 \\ [0, 0, 0, \quad -x, -y, -1, \quad y_p x, y_p y, y_p] \quad h &= 0 \end{aligned}$$

The third row of the cross product is linearly dependent on the first two. Stacking the independent rows gives $A h = 0$. The solution is the right singular vector of A associated with the smallest singular value, which minimizes $\|A h\|$ subject to $\|h\|=1$.

Why normalize points?

If source coordinates are near 2000 pixels while destination coordinates are near 20, entries in A can differ greatly in scale. Hartley normalization translates each point set to its centroid and scales the mean Euclidean distance to $\sqrt{2}$. Solve in normalized coordinates, then undo the coordinate transforms.

```
centroid = mean(points)
mean_distance = mean(norm(point-centroid))
scale = sqrt(2) / mean_distance
T = [s 0 -s*cx; 0 s -s*cy; 0 0 1]
H_pixel = inverse(T_dst) * H_normalized * T_src
```

The final normalization $H[2,2]=1$ is a convenient scale convention. It is unsuitable when that element is close to zero; a more general norm-based scale would be needed for that case. Degenerate samples, such as four collinear points, do not define a stable projective transform.

11. RANSAC: consensus and confidence

RANSAC separates the model-fitting problem from correspondence cleaning. It assumes that a sufficiently large subset of pairs agrees with one model and that random minimal samples eventually contain only inliers.

```
repeat for each hypothesis:
    sample 4 distinct correspondence indices
    estimate H with normalized DLT
    compute e_i = norm(project(H*p_i) - pprime_i) for every pair
    inlier_i = (e_i < epsilon)
keep the hypothesis with the largest inlier count
refit H from all inliers; re-evaluate the final inlier mask
```

Iteration count intuition

If the inlier fraction is w and a sample contains $s=4$ pairs, the all-inlier probability is w^s . The probability of failing all R independent trials is $(1-w^s)^R$. Requiring failure probability at most $1-p$ gives the familiar bound below.

$$P(\text{success by } R) = 1 - (1 - w^4)^R$$

$$R \geq \log(1-p) / \log(1-w^4)$$

For example, with $w=0.5$, a four-point sample is entirely inlier with probability $1/16$. At least 72 independent samples are needed for roughly 99 percent probability under the ideal model. Real samples are not perfectly independent, and degeneracy lowers the usable-sample probability.

The inlier threshold ϵ is measured in destination pixels. Scale it with expected localization noise. Too small rejects noisy true matches; too large accepts false matches. The routine returns both a refined matrix and a Boolean inlier mask, not just a scalar quality score.

12. Harris response and Hough voting

Harris: from image shift to structure tensor

For a small displacement δ , a first-order Taylor approximation gives $I(x+\delta_x, y+\delta_y)$ approximately $I + I_x\delta_x + I_y\delta_y$. The squared change over a window is therefore a quadratic form in the displacement.

```

E(delta) = sum_window w(x,y) [I(x+delta)-I(x)]^2
           approximately delta^T M delta
M = [sum(w Ix^2), sum(w IxIy); sum(w IxIy), sum(w Iy^2)]

```

Eigenvectors of M give directions of strongest and weakest local change. Two large eigenvalues indicate a corner; one large and one small indicate an edge; both small indicate a flat patch. The determinant-trace formula avoids explicitly computing eigenvalues at every pixel.

Hough transform: lines as accumulator intersections

A line is described by its normal angle θ and signed distance ρ . Each edge point votes along its sinusoid in parameter space. Collinear points have sinusoids that intersect at their common line parameters.

```

rho = x*cos(theta) + y*sin(theta)
theta spans a half-turn; rho spans the image diagonal
A[rho_bin,theta_bin] += 1 for every edge point and angle

```

The bin grid controls the balance between localization and robustness. Peak clustering is necessary because one geometric line can distribute votes among adjacent bins. Harris and Hough are complementary: Harris returns localized two-dimensional corners; Hough aggregates many edge points into global line hypotheses.

13. Region moments: from pixels to descriptors

Region measurements begin with the foreground coordinate set C . The zeroth raw moment is pixel count; first moments give the centroid. Central moments describe spread after translation to that centroid.

```

m00 = sum 1 = A
m10 = sum x; m01 = sum y
cx = m10/A; cy = m01/A
mu20 = sum (x-cx)^2
mu02 = sum (y-cy)^2
mu11 = sum (x-cx)(y-cy)

```

The covariance-like matrix $C = (1/A)[[\mu_{20}, \mu_{11}], [\mu_{11}, \mu_{02}]]$ is symmetric and positive semidefinite. Its eigenvalues measure squared spread along principal axes; the eigenvector associated with the larger eigenvalue estimates the major axis.

```

phi = 0.5 * atan2(2*mu11, mu20-mu02)
eccentricity = sqrt(1-lambda2/lambda1), lambda1 >= lambda2

```

Area, extent, perimeter, and solidity

Pixel area counts foreground pixels; box area counts all cells in the enclosing rectangle. Extent is their ratio. Perimeter is estimated by a boundary chain code, and circularity compares area with squared perimeter. Solidity compares pixel area with the convex hull.

```

extent = pixel_area / bbox_area
circularity = min(1, 4*pi*pixel_area / perimeter^2)
solidity = pixel_area / convex_hull_area

```

A filled axis-aligned rectangle has extent one. A circle fills $\pi/4$, about 0.785, of its continuous bounding square. Rasterization, holes, narrow structures, and background leakage change these values. Descriptors are measurements of a binary mask, not invariants of the underlying continuous object.

14. NCC: invariance and implementation

Normalized cross-correlation measures how patch deviations co-vary with template deviations. Flatten both arrays and denote centered vectors p and t . The score is their normalized inner product.

```
p = vec(P - mean(P)); t = vec(T - mean(T))
NCC = dot(p,t) / (norm(p)*norm(t))
```

Affine brightness invariance

Adding a constant to a patch vanishes after mean subtraction. Multiplying a patch by a positive contrast scale multiplies both its centered values and its standard deviation by the same amount, which cancels in the ratio. Multiplication by a negative scale reverses the score sign.

```
Pprime = a*P + b, a>0
Pprime - mean(Pprime) = a*(P - mean(P))
NCC(Pprime,T) = NCC(P,T)
```

The score is insensitive to a global brightness offset and positive gain, but not to rotations, shape changes, occlusion, spatial misalignment, or local nonlinear illumination. Near-zero standard deviation leaves no discriminative pattern; the implementation defines that score as zero.

Memory and search size

The `as_strided` patch tensor is a view. Computing centered patch arrays and their products materializes large temporaries. If the output map has M locations and each template has q pixels, direct work and these temporaries scale as $M*q$. A memory guard estimates the largest temporary and fails before an unsafe allocation.

15. Kalman filter: prediction, correction, gating

The state combines measured position with latent velocity. A linear Gaussian model describes motion uncertainty and centroid noise. The filter alternates prediction and correction.

```
s = [x,y,vx,vy]^T
F(dt) = [1 0 dt 0; 0 1 0 dt; 0 0 1 0; 0 0 0 1]
H = [1 0 0 0; 0 1 0 0]
state_pred = F*state; P_pred = F*P*F^T + Q
```

The measurement z contains only x and y . Innovation $r=z-H*state_pred$ says how far the observation differs from prediction; $S=H*P_pred*H^T+R$ expresses uncertainty in that difference. The gain balances prior and measurement uncertainty.

```
K = P_pred*H^T*inverse(S)
state = state_pred + K*r
P = (I-KH)P_pred(I-KH)^T + K R K^T
```

Interpreting noise

Increasing R makes the filter trust measurements less and smooth more strongly; increasing Q allows greater deviation from constant velocity. A variable frame interval belongs in $F(dt)$. The implementation defaults to a diagonal Q and R , which is compact but only a rough motion/noise model.

Gating and occlusion

Squared Mahalanobis distance scales residual by innovation uncertainty. A two-dimensional threshold near 9.21 corresponds approximately to the 99th percentile under a calibrated Gaussian model. When a measurement is rejected, the prediction remains committed. During a missing observation, call predict once for each elapsed frame; on a measured frame, call update.

Each object has a filter and persistent ID. Greedy nearest-pair association sorts track-detection distances and accepts the closest eligible pairs under a distance limit. New unmatched detections create IDs; tracks with repeated misses are eventually removed. This simple method can be ambiguous when objects cross or cluster.

16. CNN layers and gradient equations

Dense layer

```
Z = X W^T + b; delta = dL/dZ
dW = delta^T X; db = sum_batch(delta)
dX = delta W
```

Backpropagation applies the chain rule in reverse layer order. ReLU multiplies the upstream derivative by the indicator that its cached input was positive. Flatten is a reshape in the forward pass and the inverse reshape in the backward pass.

Cross-correlation convolution

```
Y[n,o,y,x] = b[o] + sum(c,i,j) X[n,c,y*s+i,x*s+j] W[o,c,i,j]
dW[o,c,i,j] = sum(n,y,x) delta[n,o,y,x] X[n,c,y*s+i,x*s+j]
db[o] = sum(n,y,x) delta[n,o,y,x]
```

Unlike `filters.convolve2d`, CNN convolution follows the standard deep-learning convention and does not flip the weights. Max pooling sends gradient to the maximum; exact ties split it evenly. Average pooling divides each incoming gradient by the window area.

Softmax cross-entropy and checking

```
p[c] = exp(z[c]-max(z)) / sum_j exp(z[j]-max(z))
L = -mean(log(p[target]))
dL/dz = (p - one_hot(target)) / batch_size
g_numeric[j] = (L(theta[j]+eps)-L(theta[j]-eps))/(2*eps)
```

Subtracting the largest logit stabilizes exponentiation. Centered finite differences provide a numerical check of analytic backpropagation away from nondifferentiable ReLU and pooling ties. The included tests exercise these layer gradients. The optimizer is plain SGD, not an adaptive optimizer.

17. Shape CNN: architecture and data scope

```
Input: binary 24x24 mask
Conv 1->8, 3x3, same; ReLU; MaxPool 2 -> 8x12x12
Conv 8->16, 3x3, same; ReLU; MaxPool 2 -> 16x6x6
Flatten 576; Dense 576->32; ReLU; Dense 32->3
```

Preprocessing extracts the tight nonzero bounding box, scales the longer side to 24 while preserving aspect ratio, centers on a zero canvas, bilinearly samples, then thresholds at 0.5. This retains rectangle elongation; warping height and width independently would destroy that cue.

The model was trained on synthetic circles, squares, and rectangles with size and position jitter, rotation, and degradation. Version 4.2.1 fixed incomplete square rotation coverage and moved degradation augmentation before resizing. The metadata reports 99.17 percent clean synthetic validation accuracy and 96 percent on a combined stress set.

A recorded weak case is 3 percent uniform salt-and-pepper flips: 85 percent overall and 65 percent on circles. These figures describe those generated distributions. Real segmentation masks may contain different artifacts, so confidence is not automatically calibrated for a camera or deployment.

The classifier outputs three class probabilities. A blank mask is rejected because none of the classes means absence of an object. The geometric heuristic remains useful as an independent second opinion, especially when CNN confidence is low.

18. Worked pipeline and parameter checklist

A practical shape-tracking pipeline keeps localization, segmentation, measurement, and temporal filtering distinct. Each stage consumes a different representation and has different failure modes.

```
RGB frame -> HSV threshold -> binary mask
mask -> opening/closing -> connected components
component -> area filter, exclusive box, centroid, metrics
cropped mask -> optional shape CNN
centroid detections -> multi-object association and Kalman filters
```

Reason about stage boundaries

If components are not split, the centroid of two objects can land between them. If shape descriptors are computed from a fixed template rectangle, they describe the window rather than the object. Use a detector or colour mask to locate a region, then measure the segmented component inside that region.

Tune each parameter in its own units

- Motion threshold: grayscale intensity difference in 0..255.
- NCC threshold: dimensionless correlation score in -1..1.
- NMS threshold: dimensionless IoU in 0..1.
- RANSAC threshold: reprojection error in destination pixels.
- Kalman gate: squared Mahalanobis distance, not pixels.
- Minimum component area: foreground pixels; scale it when image resolution changes.

Evaluate parameters on representative frames. The visible performance of the whole pipeline is limited by its weakest stage: a precise tracker cannot recover an object missing from the segmentation mask, and a confident shape model cannot correct a crop that contains the wrong region.

19. Formula index and reproducibility notes

```
Luminance:  $Y = .299R + .587G + .114B$ 
Threshold:  $T_{\tau}(I) = 255 * 1[I \geq \tau]$ 
Histogram map:  $L(k) = \text{round}(255 * \sum_{j \leq k} p(j))$ 
Sobel:  $M = \sqrt{G_x^2 + G_y^2}$ 
Harris:  $R = \det(M) - k \text{trace}(M)^2$ 
Line:  $\rho = x \cos(\theta) + y \sin(\theta)$ 
NCC: centered cosine similarity
```

```
IoU: intersection / union
Kalman: state <- state_pred + K*(z-H*state_pred)
```

For exact function signatures, accepted shapes, defaults, and return keys, consult the v4.2.1 module docstrings. The unit-test suite records executable examples of numerical conventions, including finite-difference checks for neural-network gradients. The pretrained model and its metadata are distributed as package data.

This document describes the current implementation and its assumptions. It does not replace empirical evaluation on the images, cameras, noise levels, and runtime constraints relevant to a particular application.

20. Function-by-function API guide

This index records the purpose and output role of the exported operations. It is an orientation guide; the Python docstrings remain the authority for exact shapes, defaults, and validation behavior.

Core, colour, and filters

`rgb_to_grayscale` maps RGB bytes to grayscale. `threshold` makes the inclusive 0/255 mask. `histogram_equalize` builds and applies a cumulative-distribution lookup table. `rgb_to_hsv` returns float32 hue in degrees and normalized saturation/value. `convolve2d` computes true convolution with same or valid padding. `sobel_edge_detection` returns raw gradients, normalized magnitude, and direction; `SOBEL_X` and `SOBEL_Y` expose its kernels.

Morphology and geometry

`dilate` and `erode` apply binary ANY/ALL neighborhoods; `opening` and `closing` compose them in opposite orders. `grayscale_dilate` computes a sliding maximum. `label_connected_components` labels regions; `component_properties` measures each label; `select_component` chooses by area or proximity. `bilinear_interpolate` samples a grayscale plane. `rotate_image` rotates around image center. `warp_perspective` applies a homography by inverse destination sampling.

Features and detection

`harris_corner_response` computes the response map; `detect_harris_corners` thresholds and suppresses local peaks. `hough_line_transform` returns the accumulator hypotheses; `count_hough_lines` clusters peaks. `convex_hull` computes a 2D monotone-chain hull. `extract_object_metrics` derives exclusive boxes, pixel area, moments, perimeter, and shape descriptors. `draw_bounding_box` modifies an RGB image; `classify_object` applies nearest-reference colour and geometric shape rules.

`match_template_ncc` returns a valid-position score map. `find_template_matches` thresholds it and applies NMS. `gaussian_pyramid` creates blur-and-decimate scales. `match_template_ncc_multiscale` searches those scales. `non_max_suppression` removes boxes whose IoU exceeds the selected threshold.

Tracking and calibration

`compute_motion_mask` differences two grayscale frames in widened signed arithmetic. `color_mask` performs HSV range selection; `color_mask_hue_wrap` handles the angular seam; `color_mask_adaptive` changes strategy for achromatic samples. `calculate_centroid` returns a mask center or None. `TemporalSmoother` averages a frame ring buffer. `kalman_filter_predict` and `kalman_filter_update` expose matrix-level operations; `mahalanobis_gate` scores an innovation. `KalmanCentroidTracker` manages one position/velocity state, and `MultiObjectKalmanTracker` manages IDs and greedy detection association.

`normalize_points` returns normalized coordinates and transform. `solve_homography_dlt` fits a projective map. `reprojection_error` returns one error per pair. `ransac_homography` robustly fits and returns a final inlier mask.

Neural networks and optional streaming

`mycv.nn` provides `Conv2D`, `MaxPool2D`, `AvgPool2D`, `Flatten`, `Dense`, `ReLU`, `Sequential`, `SGD`, and `softmax_cross_entropy`. `Conv2D` uses NCHW cross-correlation; the loss returns mean batch loss and logits gradient. `build_shape_cnn` creates an untrained fixed architecture; `load_shape_cnn` loads trained weights; `predict_shape` returns label, confidence, and class probabilities.

`StreamReader` is exposed from `mycv.streaming` and requires optional PyAV. The package intentionally does not import streaming from the root namespace so core import remains independent of PyAV.